

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.: T013	Name: Saqlain Khan
Class: TYIT	Batch: 1
Date of Assignment: 13-07-2026	Date/Time of Submission: 13-07-2026

Part B - Program

- a) Create methods `add()`, `multiply()`, `subtract()`, `divide()` with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

STEP 1 :- START

STEP 2 :- Define a delegate `Operations(int a, int b)`

STEP 3 :- Create the calculator class with `Add`, `Subtract`, `Multiply`, and `Divide` methods

STEP 4 :- Create the calculator object and delegate object

STEP 5 :- Assign the method

STEP 6 :- Display the results

STEP 7 :- STOP

Code:

```
using System;
delegate void Operation(int a, int b);
class Calculator
{
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition = " + (a + b));
    }
    public void Subtract(int a, int b)
    {
        Console.WriteLine("Subtraction = " + (a - b));
    }
    public void Multiply(int a, int b)
    {
        Console.WriteLine("Multiplication = " + (a * b));
    }
    public void Divide(int a, int b)
    {

```

```
        if (b != 0)
            Console.WriteLine("Division = " + (a / b));
        else
            Console.WriteLine("Cannot divide by zero");

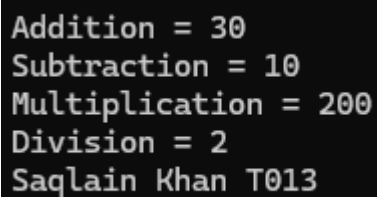
    }
}
class Program
{
    static void Main(string[] args)
    {
        Calculator c = new Calculator();
        Operation obj;
        obj = c.Add;
        obj(20, 10);

        obj = c.Subtract;
        obj(20, 10);

        obj = c.Multiply;
        obj(20, 10);

        obj = c.Divide;
        obj(20, 10);

        Console.WriteLine("Saqlain Khan T013");
        Console.ReadLine();
    }
}
```

Output:A screenshot of a terminal window with a black background and white text. The output shows the results of the calculator operations: Addition = 30, Subtraction = 10, Multiplication = 200, Division = 2, and the author's name Saqlain Khan T013.

```
Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2
Saqlain Khan T013
```

b) Write a program using multicast delegates.

Algorithm:

STEP 1 :- START

STEP 2 :- Define the delegate

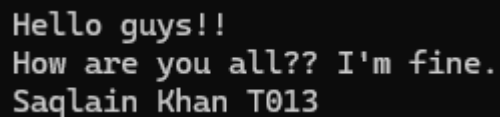
STEP 3 :- Create the methods of Hello() and Fine()

STEP 4 :- Create the object

STEP 5 :- Display the results

STEP 6 :- STOP**Code:**

```
using System;
delegate void Message();
class Program
{
    public void Hello()
    {
        Console.WriteLine("Hello guys!!");
    }
    public void Fine()
    {
        Console.WriteLine("How are you all?? I'm fine.");
    }
    static void Main(string[] args)
    {
        Program p = new Program();
        Message msg = p.Hello;
        msg += p.Fine;
        msg();
        Console.WriteLine("Saqlain Khan T013");
        Console.ReadLine();
    }
}
```

Output:A screenshot of a terminal window showing the output of the program. The text is displayed on three lines: "Hello guys!!", "How are you all?? I'm fine.", and "Saqlain Khan T013". The background is dark, and the text is light-colored.

```
Hello guys!!
How are you all?? I'm fine.
Saqlain Khan T013
```

- c) Create a class **BankAccount** with **AccountNumber** and **Balance**. Implement property for **Balance** with validation (**Balance** cannot be negative).

Algorithm:

STEP 1 :- START

STEP 2 :- Create a class named as **BankAccount**

STEP 3 :- Validate the balance so it cant be negative

STEP 4 :- Assign the account number

STEP 5 :- Display the account number and balance

STEP 6 :- STOP

Code:

```
using System;
class BankAccount
{
```

```
public int AccountNumber { get; set; }
private double balance;
public double Balance
{
    get { return balance; }
    set
    {
        if (value >= 0)
            balance = value;
        else
            Console.WriteLine("Balance cannot be negative.");
    }
}
}
class Program
{
    static void Main(string[] args)
    {
        BankAccount b = new BankAccount();
        b.AccountNumber = 1001;
        b.Balance = 5000;
        Console.WriteLine("Account Number: " + b.AccountNumber);
        Console.WriteLine("Balance: " + b.Balance);
        b.Balance = -2000;
        Console.WriteLine("Final Balance: " + b.Balance);
        Console.WriteLine("Saqlain Khan T013");
        Console.ReadKey();
    }
}
```

Output:

```
Account Number: 1001
Balance: 5000
Balance cannot be negative
Final Balance: 5000
Saqlain Khan T013
```